TECHNICAL ARCHITECTURE DOCUMENT

Egyptian Dental Clinic AI Operations Platform

A WhatsApp booking agent and staff dashboard sharing one live database — architecture, webhook design, and load-test findings.

Document type
Engineering architecture & test report

Date
2026-08-18

Scope
Backend, frontend, WhatsApp agent, webhook subsystem, load testing

Status
Local development environment (see §10)

Table of Contents

01 Executive Summary

This document describes an operations platform for a single dental clinic in Egypt, built around one principle: **the same data should drive both how the clinic is booked and how it is run.** A patient booking an appointment over WhatsApp and a secretary rescheduling that same appointment from a browser are both writing to the same Postgres tables, in real time, with no synchronization step between them.

The system has two cooperating halves. The first is a **conversational WhatsApp agent** built on LangGraph, which understands Egyptian Arabic booking requests (typed or spoken), looks patients up deterministically from their WhatsApp number, checks real availability, and books, reschedules, or cancels appointments — all without a staff member touching the conversation. The second is a **React dashboard** used by the doctor (posting procedures performed, reviewing analytics) and the secretary (running the daily schedule, collecting payments).

The most significant engineering event covered in this document is a mid-project architectural migration: the WhatsApp agent originally polled for new messages every ten seconds from inside a Jupyter notebook. It now receives messages the instant they arrive via an HMAC-signed webhook, processed inside the same FastAPI backend that serves the dashboard. That migration surfaced seven real, previously-latent bugs — including two that a k6 load test caught directly, one of which was a genuine concurrency-safety defect (a database connection shared unsafely across threads) that had not manifested under the system's original single-threaded design. This document treats that discovery process as first-class content, not an afterthought: **Sections 6 and 7** — the webhook architecture and the load-testing methodology — are the most detailed sections in this report, by design.

02 Problem Definition

A small dental clinic's operational load splits into two distinct problems that are conventionally solved with two disconnected tools — a booking channel (phone, or a generic scheduling app) and a separate practice-management system for billing and records. That split has real costs:

- **Booking requires a human to be available.** A phone must be answered during clinic hours to take a booking; outside those hours, or when the line is busy, a request is lost or delayed.
- **Two systems drift.** A booking taken over the phone has to be manually re-entered into whatever system tracks the schedule and the patient's payment history — a mechanical step that is itself a source of errors and delay.
- **No feedback loop.** Without a system that already knows the schedule and the payment history, there is no natural place to compute (let alone act on) things like "which patients haven't been in for six months" or "what did no-shows actually cost us this month."

The specific, narrower engineering problem this document focuses on — because it is where the most substantive work happened — is: **how does an automated WhatsApp agent receive a patient's message with low latency, high reliability, and safe concurrency, without either polling a gateway wastefully or introducing a fragile custom protocol?** The answer chosen and validated here is a webhook, and the validation is a real load test, not an assumption.

03 Proposed Solution

The solution is a single FastAPI backend serving two audiences from one database:

1. **An event-driven WhatsApp agent** (`app/whatsapp_agent/`), triggered by a webhook rather than a poll loop, running a LangGraph state machine with nine domain-specific tools (lookup/register a patient, check availability, book, reschedule, cancel, send a confirmation) against the same `appointments` / `patients` tables the dashboard reads.
2. **A role-gated REST API** (`app/routers/`) for the doctor and secretary dashboards, authenticated with short-lived JWTs.
3. **An in-process scheduler** (APScheduler) that runs the clinic's recurring operational logic — reminders, monthly analytics, re-engagement — without a separate worker service, appropriate to a single-clinic deployment's actual traffic volume.

This design was chosen over the alternatives for concrete, stated reasons rather than by default:

> **WHY A WEBHOOK, NOT A QUEUE OR A THIRD-PARTY BUSINESS API**
>
> A message queue (e.g. Celery + Redis) would add two always-on services for a workload this small. WhatsApp's official Business API was evaluated for richer message types (interactive lists/buttons) and deliberately deferred — it is a materially larger integration, and the self-hosted gateway already in use

(OpenWA) has no support for those message types today. A webhook against the existing gateway was the smallest change that removed the actual problem (10-second polling latency).

WHY APSCHEDULER IN-PROCESS, NOT A SEPARATE SCHEDULER SERVICE

Three cron-style jobs at single-clinic volume do not justify the operational overhead of a dedicated scheduler service. The jobs run inside the same FastAPI process, started and stopped via its lifespan hooks.

04 System Architecture

4.1 Components

COMPONENT	RESPONSIBILITY	TECHNOLOGY
OpenWA gateway	Owns the live WhatsApp session (Baileys engine); exposes a REST API for sending messages and an outbound webhook subsystem for receiving them	`Node/NestJS, Docker`
Webhook receiver	Verifies signature, dedups by idempotency key, ACKs fast, hands off to background processing	`FastAPI async route`
Dispatcher	Concurrency control — per-chat lock (serial per patient) + global semaphore (bounded parallel conversations)	`asyncio`
LangGraph agent	Conversation state machine: router, booking-info extraction, tool-calling reasoning loop	`LangGraph, LangChain`
Agent tools	Nine functions doing the actual DB reads/writes for booking logic, with full validation (double-booking guards, ownership checks, booking-window limits)	`psycopg2, direct SQL`
REST API	Dashboard-facing endpoints for auth, doctor actions, secretary actions	`FastAPI, SQLAlchemy`
Scheduler	Daily reminders, monthly analytics + email, re-engagement — no external trigger required	`APScheduler`
Database	Single source of truth for patients, appointments, payments, procedures, and cached analytics snapshots	`PostgreSQL (Supabase pooler)`
LLM/ASR provider	Conversation reasoning, structured extraction, monthly-insight generation, and voice transcription (both directions: patient voice notes and the doctor's dictated charges)	`Groq (chat + Whisper)`
Dashboard	Doctor and secretary browser UI	`React 19, Vite`

4.2 Architecture diagram

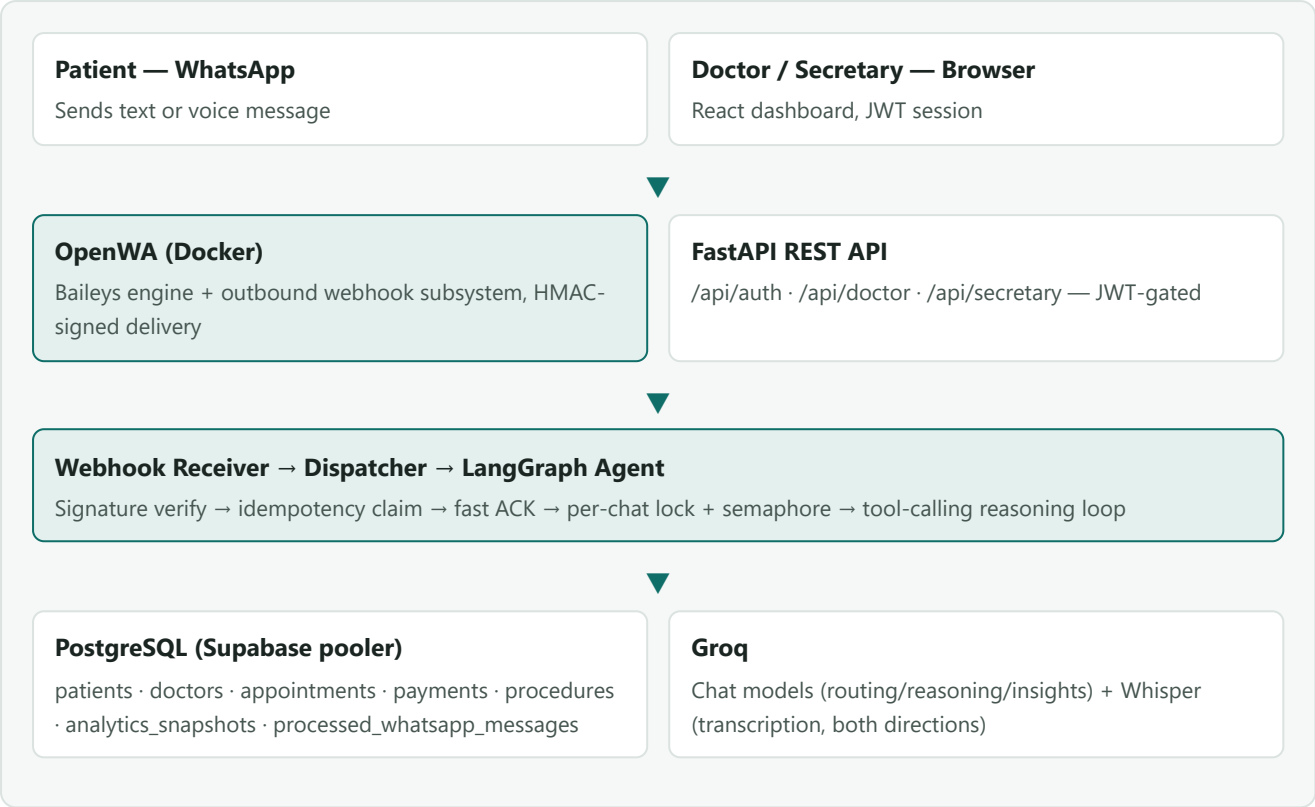

Figure 4.1 — Both the WhatsApp agent and the dashboard read and write the same Postgres database; a booking made by one is visible to the other immediately, with no synchronization step.

4.3 Internal vs. external services

INTERNAL (THIS CODEBASE)	EXTERNAL (THIRD-PARTY / VENDORED)
FastAPI backend (`app/`), including the webhook receiver, dispatcher, and agent	OpenWA — self-hosted but a separate vendored codebase, run in its own Docker container
React dashboard (`clinic_frontend/`)	Groq — third-party LLM/ASR API
Scheduler jobs, analytics engine, PDF/email reporting	Supabase — managed Postgres hosting

05 End-to-End System Flow

5.1 WhatsApp message lifecycle

Patient	Sends a WhatsApp message (text or voice)
OpenWA	Baileys engine receives it; the webhook module POSTs it to the backend, HMAC-signed, within the same request cycle — no polling interval to wait out
Backend	Verifies the signature → claims the idempotency key → returns `200` **before any LLM call happens**
	— response already sent; everything below runs in a background task —
Dispatcher	Acquires the per-chat lock and a semaphore slot, then runs the agent on a worker thread
Agent	Deterministically identifies the patient from their WhatsApp number (never asks for it); routes the message (direct reply vs. full reasoning); calls tools to check availability and book/reschedule/cancel
Agent	Sends the reply back through OpenWA's send-text API, with a candidate-based fallback across possible contact-ID forms
Patient	Receives the reply

Figure 5.1 — The full round trip. Full mechanics of the middle steps are in Section 6.

5.2 Dashboard request lifecycle

Staff user	Logs in with username/password (`POST /api/auth/login`)
Backend	Verifies the bcrypt hash, issues a 12-hour JWT (HS256) carrying the user's role
Frontend	Stores the token, attaches `Authorization: Bearer …` to every subsequent request
Backend	A `require_doctor` / `require_secretary` dependency decodes and checks the token before the route body ever runs
Backend	Reads/writes Postgres through SQLAlchemy — the identical tables the WhatsApp agent uses

Figure 5.2 — The dashboard's data-access pattern is intentionally simple: no client-side cache layer, no optimistic updates — a request always reflects the current database state.

06 Webhook Deep Dive

This is the most detailed section of this document, deliberately. It is also the section with the most direct evidence behind it: every claim below was verified live against a running OpenWA instance and a running backend during this project, not designed on paper and left untested.

6.1 What a webhook is, and why it fits here

A webhook inverts the normal client-initiates-contact model: instead of this backend repeatedly asking OpenWA "anything new?", OpenWA calls *this backend* the moment something happens. The previous design polled every ten seconds from a Jupyter kernel — every reply carried up to ten seconds of dead air, the loop died whenever the kernel restarted, and "concurrency" was an accident of the loop being single-threaded rather than a real design choice. OpenWA already ships a complete outbound-webhook subsystem (per-session registration, HMAC-signed delivery, retry with backoff), so adopting it removed the polling delay without introducing new infrastructure.

6.2 Webhook lifecycle

STAGE	WHAT HAPPENS	WHERE
Event generation	A WhatsApp message is received by OpenWA's Baileys engine	OpenWA
Delivery	OpenWA POSTs a signed JSON payload to the registered URL	OpenWA → backend
Signature verification	HMAC-SHA256 over the raw request body, constant-time compared	`app/whatsapp_agent/webhook.py`
Idempotency claim	Atomic `INSERT … ON CONFLICT DO NOTHING` against a dedicated table	Postgres
Relevance filter	Drop groups, broadcasts, newsletters, and the clinic's own outbound messages	`webhook.py`
Acknowledgement	`200 OK` returned — this is the point at which OpenWA considers delivery successful	`webhook.py`
Background processing	Voice transcription (if needed), agent reasoning, tool calls, reply send	`dispatcher.py`, `graph.py`, `sender.py`

6.3 Endpoint, headers, and payload

FIELD	VALUE / PURPOSE
Method & path	`POST /api/whatsapp/webhook`
X-OpenWA-Signature	`sha256=<hex hmac>` — computed over the raw body with the shared `WHATSAPP_WEBHOOK_SECRET`
Content-Type	`application/json`

```
{
  "event": "message.received",
  "sessionId": "…",
  "idempotencyKey": "msg_<session>_<waMessageId>_<webhookId>",
  "deliveryId": "dlv_<uuid>",
  "data": {
    "id": "…",
    "from": "201xxxxxxxx@c.us",
    "chatId": "201xxxxxxxx@c.us",
    "type": "text",            // or "voice"
    "body": "عايز احجز معاد",
    "fromMe": false,
    "isGroup": false,
    "media": { "mimetype": "audio/ogg", "data": "<base64>" }  // voice messages only
  }
}
```

Fields the backend actually consumes: `idempotencyKey` (dedup key), `event` (must equal `message.received`), and inside `data`: `chatId` / `from` (patient identity), `type` (routes text vs. voice handling), `body` (message text), `fromMe` / `isGroup` (filtering), and `media` (voice-note payload, base64-encoded audio handed directly to Groq Whisper).

6.4 Sequence diagram

Patient	Sends message via WhatsApp
OpenWA	`POST /api/whatsapp/webhook`, `X-OpenWA-Signature: sha256=…`
Backend	Verify HMAC signature over the raw body
Backend	If invalid → `401 Unauthorized`, processing stops here
Backend	`INSERT idempotency_key … ON CONFLICT DO NOTHING`
Backend	If a duplicate (0 rows inserted) → `200 {"status":"duplicate, ignored"}`, stop
Backend	`200 {"status":"accepted"}` — **the response is sent here**
	— asynchronous from this point forward —
Dispatcher	Acquire semaphore slot + per-chat lock
Agent	Transcribe (if voice) → route → extract → reason with tools → reply
Sender	POST reply to OpenWA's send-text API, candidate-based target fallback
Patient	Receives the reply

Figure 6.1 — Every branch shown here was individually verified during development: bad-signature rejection, duplicate-delivery suppression, and the full happy path, using hand-crafted signed payloads before any real WhatsApp message was sent through the pipeline.

6.5 Idempotency

OpenWA's own `idempotencyKey` is **advisory only** — OpenWA does not guarantee at-most-once delivery, and its own documentation is explicit that a queue-unavailable fallback path can double-fire a delivery. The backend therefore does not trust that key as the source of truth; it re-derives correctness itself via a database-level unique constraint (`processed_whatsapp_messages.idempotency_key PRIMARY KEY`). A retried delivery — identical key — always resolves to zero inserted rows and is treated as a no-op, returning `200` without re-running any business logic.

6.6 Timeout handling and synchronous vs. asynchronous processing

OpenWA's own delivery timeout is 10 seconds; a slower response is treated as a failed delivery and retried (up to 3 attempts, exponential backoff, per the webhook's configured `retryCount`). Because business-logic retries were already the source of a real historical bug in this project (a transient LLM failure causing the same message to be answered twice — documented in the original notebook as "FIX 22"), the backend design deliberately decouples *delivery acknowledgement* from *business-logic completion*: everything past the idempotency claim runs in a FastAPI `BackgroundTasks` callback, after the HTTP response has already been sent. This is not a performance optimization; it is what makes it safe for OpenWA's transport-level retry to exist at all without risking a duplicate reply.

6.7 Error handling within background processing

A failure inside the background task (an LLM error, a database error, a transcription failure) is caught, logged, and answered with a single generic apology to the patient. It is **not** retried automatically — retrying here would resurrect the exact double-reply failure mode the design above exists to prevent. This is a deliberate reliability trade-off: a single failed message gets one honest apology rather than a silent drop or a duplicated reply.

6.8 Security considerations

- **Signature verification** happens over the raw request bytes, before JSON parsing, using `hmac.compare_digest` (constant-time, avoiding timing side-channels).
- **SSRF protection** is enforced on OpenWA's side at webhook-registration time — it refuses to register a callback URL pointing at a loopback or private address by default. This project's local development target was added to OpenWA's explicit allow-list rather than disabling that protection wholesale.
- **Secret management:** the webhook secret lives in `.env`, distinct from the JWT signing secret and every API key, so a compromise of one credential does not cascade into the others.

6.9 Local development and Docker networking

> **A REAL BUG FOUND DURING THIS PROJECT**
>
> OpenWA runs inside a Docker container; the backend runs directly on the host machine. A webhook URL of `127.0.0.1` resolves, from inside the container, to the *container's own* loopback interface — not the host. This produced fourteen consecutive silent delivery failures (`fetch failed`) before being diagnosed. The fix was Docker Desktop's built-in `host.docker.internal` hostname, which routes from inside a container back to the host, combined with adding that hostname to OpenWA's SSRF allow-list. Verified via OpenWA's own webhook test-delivery endpoint before trusting it with real traffic.

`ngrok` is present in the repository for the case where OpenWA needs to reach a backend that is *not* on the same machine (for example, testing from a phone on a different network) — it is not required for the same-machine Docker topology described above, and was not the fix that was actually needed once the root cause was correctly diagnosed as a Docker networking issue rather than a reachability issue.

6.10 Logging and monitoring

Processing emits structured print-style log lines (message received, reply sent, send failures) to stdout. Two real, non-obvious bugs were found here during development: Windows' default console codepage could not encode the Arabic/emoji characters used throughout this logging, silently crashing the background thread on its very first line — meaning every test message was dropped with no visible error until `sys.stdout.reconfigure(encoding="utf-8")` was added at process start; and stdout was block-buffered whenever not attached to a real terminal, meaning log

lines could sit invisible indefinitely until `line_buffering=True` was added alongside it. Both are documented under Section 12 (Observability) as current gaps: there is no structured logging framework or external log aggregation today, only these corrected stdout writes.

07 k6 Load Testing

7.1 What k6 is, and why it was chosen

k6 (Grafana Labs) is a scriptable HTTP load-testing tool: a JavaScript file defines virtual users and their behavior, and k6 reports latency percentiles, throughput, and error rates under real concurrent load. It was chosen here because the alternative was accepting an untested assumption — the webhook handler *looked* non-blocking by inspection; only a real concurrent load test proved that it was not.

7.2 Load testing vs. stress testing vs. spike testing

TEST TYPE	GOAL	USED HERE?
Load testing	Measure behavior under an expected, bounded level of concurrent traffic	`IMPLEMENTED` — this is the test described below
Stress testing	Increase load until the system breaks, to find the actual ceiling	`NOT PERFORMED` — deliberately bounded to avoid exhausting Groq's real free-tier quota (see §7.4)
Spike testing	A sudden, short burst far above normal load	`NOT PERFORMED`

7.3 Core k6 concepts, as applied in this project

- **Virtual users (VUs)** — independent simulated clients executing the script concurrently. This test used up to 25.
- **Executor / scenario** — `shared-iterations`, chosen specifically because it fixes the *total* number of requests (45) while still ramping concurrency up to the VU count, rather than an open-ended `ramping-vus` stage that could generate an unbounded number of real Groq API calls.
- **Checks** — per-request assertions (HTTP 200, response body `status: "accepted"`) that don't fail the whole run, only get counted.
- **Thresholds** — pass/fail criteria for the whole run: `p(95)<2000` ms on ACK latency, `count==0` on rejected requests. A threshold breach is what makes a load test a real test rather than a benchmark with no verdict.
- **Percentiles (p90/p95/p99)** — the latency N% of requests were faster than. The 95th percentile matters more than the average here because it exposes tail behavior an average hides — a handful of very slow requests during a concurrent burst are exactly what would trip OpenWA's real 10-second delivery timeout.

7.4 The actual test script

```
export const options = {
  scenarios: {
    webhook_burst: {
      executor: 'shared-iterations',
      vus: 25,            // peak concurrency
      iterations: 45,      // fixed total volume for the whole run
      maxDuration: '90s',
    },
  },
  thresholds: {
    webhook_ack_latency: ['p(95)<2000'],
    webhook_rejected: ['count==0'],
  },
};
```

Each iteration builds a properly HMAC-SHA256-signed `message.received` payload with a **unique `chatId` and `idempotencyKey`** — every request represents a distinct simulated patient conversation, so the test genuinely exercises cross-conversation concurrency (the dispatcher's semaphore) rather than repeatedly hitting one dedup key. The message body is a plain greeting, deliberately kept on the agent's cheapest code path (one Groq classification call, no multi-step booking flow) to keep per-request cost predictable.

> **WHY 45 REQUESTS, NOT THOUSANDS**
>
> Every new synthetic chat costs one real Groq API call. Groq's free tier caps around 30 requests/minute; 45 total requests was enough to comfortably exceed both that ceiling and the application's own concurrency semaphore within a single run, without spending a third party's rate-limit budget for no additional signal. This is a resource-conscious test design choice, not a limitation of k6 itself.

7.5 Running the test

```
winget install -e --id GrafanaLabs.k6
cd load-tests
k6 run -e WEBHOOK_SECRET=<WHATSAPP_WEBHOOK_SECRET from .env> webhook_load_test.js
```

7.6 Results

The first run **failed its own threshold** — and in doing so, caught a genuine production-grade bug: the idempotency-claim query ran synchronous `psycopg2` I/O directly inside the `async def` route handler, so every concurrent request queued behind the same blocked asyncio event loop instead of running independently.

METRIC	BEFORE FIX	AFTER FIX
p95 ACK latency	8.69 s FAIL	1.61 s PASS
Max ACK latency	9.62 s	1.84 s
Avg ACK latency	4.47 s	1.32 s
Total wall time (45 req)	11.1 s	3.3 s
Throughput	4.06 req/s	13.47 req/s
Accepted / rejected	45 / 0	45 / 0

Fix: wrapping the blocking claim in `asyncio.to_thread(...)` so it executes off the event loop, exactly mirroring the pattern already used for the agent's own background processing.

7.7 Interpreting the results — worked examples

OBSERVATION	WHAT IT MEANS
High p95, low error rate	The system is *slow*, not *broken* — exactly the "before fix" row above: every request eventually succeeded (0 rejected), but a growing share of them queued dangerously close to a real downstream timeout.
High error rate	Would indicate the endpoint itself is failing under load (crashes, connection pool exhaustion, 5xx responses) — not observed in either run here.
Throughput rising while latency also rises	A classic saturation signal: the system is accepting more concurrent work than it can actually finish promptly, and a queue is silently building somewhere. In this case, the queue was the asyncio event loop itself, blocked on a synchronous DB call.
How the bottleneck was identified	By elimination: the fix that resolved the threshold failure touched exactly one blocking call in the request path, isolating the event loop as the actual constraint rather than the database, Groq, or OpenWA.

7.8 What this test does and does not measure

This test measures the receiver's own concurrency ceiling — the ACK layer — not Groq's or OpenWA's limits, which sit one layer downstream. Those were only observed indirectly: across both runs (90 real Groq calls total), zero rate-limit errors surfaced from Groq, reported here as an honest observation rather than a claim that Groq's published ~30 requests/minute limit does not apply — the application's own 5-way semaphore plus real LLM response latency naturally spaces calls out under this specific load shape. A dedicated test of Groq's own rate-limit behavior, or of the full agent pipeline's sustained throughput, was not performed and is noted as a gap in Section 16.

08 Reliability & Failure Handling

SCENARIO	CURRENT BEHAVIOR
Webhook delivery fails in transit	OpenWA retries up to 3 times with exponential backoff. If the backend was reachable and processed it, the idempotency table prevents any duplicate effect regardless of how many retries land.
Business logic fails mid-processing (LLM error, DB error)	Caught inside the background task; the patient receives one generic apology; not retried (see §6.7).
Duplicate webhook delivery	The idempotency claim (§6.5) makes this a guaranteed no-op — verified directly with an identical replayed payload during testing.
Backend process restarts	The webhook registration self-heals on every startup (create-or-update against OpenWA); in-flight background tasks from before the restart are lost, a known gap (§16).
OpenWA/Docker container is recreated or restarted	WhatsApp session auth and all webhook registrations persist in a named Docker volume and survive a container recreation; session auto-reconnect on startup was a fix made during this project (previously the relevant env var was not actually passed through docker-compose).
Network connectivity between OpenWA and the backend fails	Manifests as OpenWA's own `fetch failed` delivery-failure log; no message reaches the backend at all. This exact scenario occurred during development (§6.9) and is queryable via OpenWA's `GET /webhooks/delivery-failures` endpoint.
Concurrent load exceeds the semaphore	Excess conversations queue for a semaphore slot; the ACK layer itself remains fast and unaffected (post-fix), so OpenWA never sees a timeout even if agent processing is queued.
Two messages from the *same* patient arrive close together	Serialized by the per-chat `asyncio.Lock` — the second is guaranteed not to start until the first's reply has been sent, preserving the LangGraph thread's conversational state.

09 Security Architecture

AREA	IMPLEMENTATION
Dashboard authentication	JWT (HS256), 12-hour expiry, bcrypt password hashing (`passlib`), role claim checked by `require_doctor` / `require_secretary` FastAPI dependencies on every gated route
Webhook authentication	HMAC-SHA256 over the raw request body, constant-time comparison, verified before JSON parsing
Idempotency / replay protection	Database-backed unique constraint — a replayed webhook delivery can never trigger the agent twice
Secrets	All credentials in `.env`, never committed; distinct secrets per concern (JWT vs. webhook HMAC vs. each API key)
SSRF protection	Enforced by OpenWA at webhook-registration time; the local target is explicitly allow-listed rather than the protection being disabled
CORS	`allow_origins=["*"]` — appropriate only for local development; `MUST CHANGE` before any public deployment
Backend API rate limiting	`NOT IMPLEMENTED` on the FastAPI side. OpenWA's own inbound API is self-rate-limited (100 req/60s), but nothing gates the dashboard REST API itself today.
Uploaded receipt files	Served as static files under `/uploads` with no auth check on the static route itself; access currently relies on unguessable generated filenames rather than an authorization check — `RECOMMENDED IMPROVEMENT`
Input validation	Pydantic models on every FastAPI request body; the agent's own tools additionally validate ids, dates, and time slots defensively against a misbehaving or hallucinating LLM before any database write

10 Deployment Architecture

Current state: local development only. There is no Dockerfile for the FastAPI backend, no CI/CD pipeline, and no hosting configuration for the frontend beyond a Vite dev server. This is stated plainly rather than implied, per this document's own standard of not overclaiming implementation status.

CURRENT — LOCAL

- Backend: `uvicorn --reload` on the developer's machine
- OpenWA: Docker container on the same machine
- Webhook target: `host.docker.internal`
- Frontend: Vite dev server
- Database: already externally hosted (Supabase)
- CORS: wide open (`*`)

RECOMMENDED — PRODUCTION

- Backend: containerized, run behind a production ASGI process manager, not `--reload`
- A real public HTTPS URL for `BACKEND_PUBLIC_URL` ; OpenWA's SSRF allow-list updated to match
- Frontend: static build served from a CDN/host, with the API base URL made environment-configurable (currently hardcoded in `src/api.js`)
- Database: unchanged (already production-appropriate)
- CORS restricted to the real frontend origin
- Secrets management appropriate to the hosting platform, replacing the flat `.env` file

11 Performance & Scalability

11.1 Measured performance

From the k6 run in Section 7: p95 ACK latency of 1.61 s and throughput of 13.47 req/s at 25 concurrent virtual users, against a local backend on the same machine as OpenWA. Dashboard REST API latency under load has not been separately measured.

11.2 Where the concurrency ceiling actually sits

Application semaphore	Caps concurrent LLM-bearing conversations — the first limit any real burst of traffic hits	`WHATSAPP_AGENT_CONCURRENCY = 5`
Groq free tier	Per model, rolling window	`~30 req/min · ~1,000 req/day`
OpenWA delivery concurrency	Concurrent outbound webhook deliveries; 10 s timeout each	`10 concurrent`
OpenWA inbound API	Self-rate-limited, governs the backend's own calls to OpenWA (sends, contact lookups)	`100 req / 60 s`

Figure 11.1 — After the async-DB fix, the webhook receiver itself is no longer the bottleneck; throughput is now governed by this stack, from top to bottom.

11.3 Scaling considerations

- **Horizontal scaling** of the backend is not currently exercised by anything in this codebase — the semaphore and per-chat locks are in-process (Python `asyncio` primitives), so running multiple backend instances would require moving that coordination into something shared (e.g. Redis-backed locks) rather than assuming a single process owns all conversations.
- **Vertical scaling** (a faster/larger machine) would raise the practical ceiling of `WHATSAPP_AGENT_CONCURRENCY` but not the Groq or OpenWA limits below it.
- **Async processing** is already used throughout the webhook path (§6.6); the dashboard REST API is synchronous request/response by design, appropriate to its current usage pattern.
- **A real task queue** (Celery/Redis, or similar) is a reasonable future step if message volume ever outgrows the in-process semaphore/lock model — not needed at the clinic's current single-location scale.
- **Caching** already exists at one layer: past (closed) months' analytics are cached in `analytics_snapshots` rather than recomputed on every dashboard view; the current month is always computed fresh.

12 Observability

CAPABILITY	STATUS
Application logs (stdout)	`IMPLEMENTED` — UTF-8 and line-buffering fixed during this project (§6.10) so Arabic/emoji log lines no longer crash the process and appear in real time rather than sitting in a buffer
Structured logging (JSON logs, log levels)	`NOT IMPLEMENTED` — current logs are plain `print()` statements
Error tracking (e.g. Sentry)	`NOT IMPLEMENTED`
Request tracing / correlation IDs	`NOT IMPLEMENTED` — OpenWA's `deliveryId` / `idempotencyKey` exist per-webhook-delivery but are not currently propagated into a formal trace
k6 metrics	`IMPLEMENTED` — ad hoc, run manually (§7), not wired into continuous monitoring
Health checks	`PARTIAL` — `GET /` returns a static ok status; OpenWA has its own `/api/health/ready`, used during development to gate startup checks
Failed-send audit trail	`IMPLEMENTED` — a dedicated `failed_whatsapp_sends` table logs chat id, message text, and timestamp for any send the candidate-based sender could not deliver

Recommendation: the failed-delivery and failed-send tables already give a durable audit trail; the highest-value near-term addition would be alerting on non-empty results from those tables, rather than building a full observability platform for a single-clinic deployment.

13 Repository Structure

```
app/
├── main.py                  # Wiring: lifespan, routers, CORS, UTF-8 stdout fix
├── models.py                 # SQLAlchemy models
├── database.py               # Engine/session (Postgres, Supabase pooler)
├── auth.py                   # JWT + bcrypt + role dependencies
├── scheduler.py                # APScheduler jobs
├── analytics.py                 # Revenue/no-show/retention analytics + LLM insights
├── reports.py / email_sender.py  # Monthly PDF + email delivery
├── voice_charge.py                # Doctor's voice-to-charge pipeline
├── whatsapp.py                     # Proactive (non-conversational) sends
├── receipts.py / config.py          # File storage; clinic-wide constants
├── routers/                          # auth.py · doctor.py · secretary.py
└── whatsapp_agent/                    # Event-driven booking agent
    ├── webhook.py                       # Receiver: signature, dedup, ACK
    ├── dispatcher.py                      # Concurrency control
    ├── handler.py                          # Payload → agent handoff
    ├── graph.py                             # LangGraph state machine
    ├── tools.py                              # 9 booking-domain tools
    ├── sender.py                              # Candidate-based reply sender
    ├── transcription.py                        # Voice → text
    ├── db.py                                    # Thread-local psycopg2 connections
    ├── llm.py / config.py                        # Model clients, shared constants
    └── registration.py                            # Self-registers the webhook

clinic_frontend/src/
├── pages/            # Login, DoctorView, SecretaryView (+ sub-views)
├── components/         # AppShell, Sidebar, TopBar, DateStrip, StatTile, TrendChart
├── auth/AuthContext.jsx  # JWT persistence + role hydration
└── api.js                  # fetch-based API client

OpenWA/                # Self-hosted WhatsApp gateway (separate Node/NestJS project)
load-tests/webhook_load_test.js   # k6 script (Section 7)
add_*.py                           # Idempotent DB migrations (no Alembic)
agent_Start.ipynb                    # WhatsApp agent notebook (reference only, not live)
```

14 Setup & Configuration

14.1 Prerequisites

- Python 3.14+ with uv
- Node.js (frontend and OpenWA)
- Docker Desktop (OpenWA)
- A Postgres database (this project uses Supabase's transaction pooler)
- A Groq API key (required); SMTP credentials (optional, monthly email only)

14.2 Key environment variables

VARIABLE	PURPOSE
`DATABASE_URL`	Postgres connection string (Supabase transaction pooler, port 6543)
`JWT_SECRET`	Signs dashboard login tokens
`GROQ_API_KEY`	LLM and Whisper calls
`OPENWA_URL / OPENWA_API_KEY / OPENWA_SESSION_ID`	Talking to the WhatsApp gateway
`WHATSAPP_WEBHOOK_SECRET`	HMAC key for verifying inbound webhook deliveries
`WHATSAPP_AGENT_CONCURRENCY`	Max concurrent conversations processed at once (default 5)
`BACKEND_PUBLIC_URL`	Where OpenWA should reach the backend (`http://host.docker.internal:8001` locally)

Present in `.env` but not currently exercised by the active code path: `DEEPSEEK_API_KEY`, `OPEN_ROUTER`, `HF_TOKEN`, `TWILIO_*` — remnants of earlier LLM/voice-provider experimentation. Confirm relevance before relying on any of these.

14.3 Install and run

```
# Install
uv sync
cd clinic_frontend && npm install
cd OpenWA && docker compose up -d

# Migrate (idempotent, run once, in any order)
.venv\Scripts\python.exe add_doctor_user_link.py
.venv\Scripts\python.exe add_receipt_column.py
.venv\Scripts\python.exe add_reminders_and_analytics.py
.venv\Scripts\python.exe add_whatsapp_opt_in.py
```

```
.venv\Scripts\python.exe add_whatsapp_agent_state.py
.venv\Scripts\python.exe seed_users.py

# Run (three processes)
cd OpenWA && docker compose up -d
.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8001 --reload
cd clinic_frontend && npm run dev -- --port 5173
```

15 Testing Strategy

LAYER	METHOD	STATUS
Webhook signature / idempotency / filtering	Manual signed-payload requests during development	VERIFIED
Webhook concurrency	k6 load test, 25 VU / 45 req (Section 7)	VERIFIED
End-to-end booking flow	Real WhatsApp messages against a live OpenWA session	VERIFIED MANUALLY
REST API (auth/doctor/secretary)	Manual HTTP testing during feature development	EXERCISED, not automated
Unit tests	—	NOT IMPLEMENTED
Integration tests	—	NOT IMPLEMENTED
Frontend component tests	—	NOT IMPLEMENTED
Stress / spike testing	—	NOT PERFORMED (§7.2)

The honest summary: this project's testing to date is **real but manual and targeted** — every claim in this document about correctness was verified against a running system at least once, but there is no regression-preventing automated suite yet. That gap is the single highest-value addition recommended in Section 17.

16 Limitations

- The WhatsApp agent uses a raw `psycopg2` connection (thread-local, autocommit) architecturally separate from the dashboard's SQLAlchemy session — two data-access patterns in one codebase.
- `sqlalchemy` is imported directly by `app/database.py` / `app/models.py` but is not listed in `pyproject.toml`'s own dependency list (only in the older `requirements.txt`) — should be resolved explicitly rather than relying on a transitive install.
- Monthly PDF report generation depends on a Windows-only font path (`C:\Windows\Fonts\tahoma.ttf`) for Arabic rendering — would need a bundled TrueType font before any non-Windows deployment.
- No automated test suite (Section 15).
- No structured logging, error tracking, or metrics platform (Section 12).
- CORS is wide open and there is no backend-level API rate limiting (Section 9).
- In-process concurrency primitives (the semaphore and per-chat locks) do not survive a process restart and would not coordinate correctly across multiple backend instances (Section 11.3).
- The booking agent currently hardcodes a single doctor (`doctor_id=1`) — the data model already supports multiple doctors, but the agent's conversational logic does not yet route between them.
- k6 testing measured the webhook receiver's ACK layer specifically; the full agent pipeline's sustained throughput under load, and Groq's real rate-limit behavior under deliberate saturation, were not separately tested (§7.8).

17 Future Roadmap

17.1 Near-term (directly addresses a listed limitation)

- A pytest suite covering the router layer and the agent's tool functions.
- A backend Dockerfile and an actual production deployment target.
- Structured logging plus basic request tracing/health metrics.
- Unify the two database-access patterns in the WhatsApp agent.
- Backend-level API rate limiting and CORS restricted to a real origin.

17.2 Longer-term

- Multi-doctor / multi-clinic support, extending the agent's routing logic to match the data model's existing capability.
- A real task queue (Celery/Redis or similar) if message volume outgrows the in-process semaphore/lock model.
- Interactive WhatsApp message types (lists/buttons) — evaluated and deliberately deferred during this project, since the current self-hosted gateway has no support for them and the official WhatsApp Business API is a substantially larger integration.
- Dedicated stress/spike testing of the full agent pipeline, and of Groq's real rate-limit behavior under deliberate saturation, as a follow-up to the load testing in Section 7.

18 Final Architecture Summary

One Postgres database sits at the center of two cooperating systems. A patient's WhatsApp message reaches the backend through a signed, deduplicated, fast-acknowledging webhook rather than a poll loop; a per-chat lock and a bounded semaphore let independent conversations run concurrently without corrupting shared state; and a LangGraph agent reasons over nine domain tools to actually book, reschedule, or cancel against the live schedule. The doctor and secretary work the same schedule and the same payment records from a React dashboard, authenticated by short-lived JWTs, with a scheduler quietly running reminders and re-engagement in the background. Every architectural claim about the webhook path in this document was verified against a running system, and the concurrency claims specifically were verified under real load with k6 — which is how a genuine, otherwise-invisible concurrency bug was found and fixed before it could affect a real patient.